



TOSHKENT SHAHRIDAGI INHA UNIVERSITETI
INHA UNIVERSITY IN TASHKENT

STUDENT NAME: Zavohirbek Xatamov

STUDENT ID NUMBER: U2210251 TION NUMBER IN THIS COURSE: 003

Homework1 – SPRING 2024

COURSE NAME: Computer Architecture

COURSE NUMBER: SOC2060

EXAMINATION DATE: 26/03/2024 TIME:

EXAMINATION DURATION:

ADDITIONAL MATERIALS
ALLOWED TO USE: None

SPECIAL INSTRUCTIONS: - Answers will only be evaluated if they are readable.
- Show all your work. For some questions, you may get partial credit even if the end result is wrong due to a calculation mistake. If you make assumptions, state your assumptions clearly and precisely.

Please do not open the examination paper until directed to do so.

READ INSTRUCTIONS FIRST:

Desks should be free from all unnecessary items (books, notes, technology, food, water, clothes)

Use of any electronic device (Phone, iPod, iPad, laptop) is not allowed during the examination.

Cheating, talking to fellow students, singing, turning back are not allowed

Write your Name (*capital letters*), ID number and Section number in each page of your examination paper.

Final answers must be written by **only blue or black, non-erasable pen**. Do not use highlighters or correction pen.

All answers should be written in the space provided for each question, unless specified the other way.

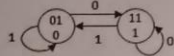
If you have a problem please raise your hand and wait quietly for a Proctor.

You are not allowed to leave the exam room until you submit the exam papers.

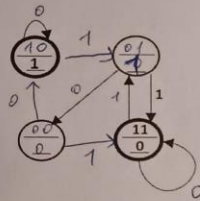
STUDENT NAME: Yashvirbux Tamwar
 STUDENT ID NUMBER: 4226251 SECTION NUMBER IN THIS COURSE: 003

1.[12 points] Finite State Machine

(a)[12 points] Given the partially completed truth table and FSM diagram below, complete all the missing entries in the truth table and the FSM diagram. In each state circle, the top entry is S_1S_0 and the bottom entry is the value of Out. Make sure that you have labeled all missing states, inputs and outputs, and that you have added and labeled any missing transitions in the FSM.



Note that in the above diagram, '01' and '11' in the circle are encoded states, and '0' and '1' in the circle are outputs. In state '01', if input is '0', next state is '11' and output is '0'. In state '01', if input is '1', next state is '01' and output is '0'.

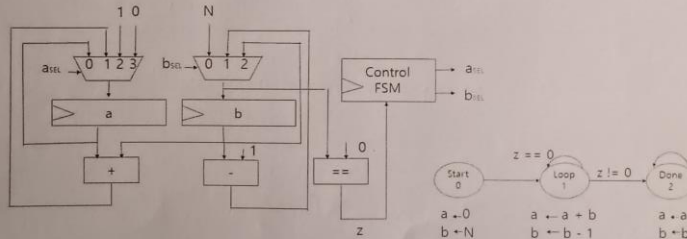


S_1	S_0	In	S_1'	S_0'	Out
0	0	0	1	0	0
0	0	1	1	1	0
0	1	0	0	0	1
0	1	1	1	1	1
1	0	0	1	0	1
1	0	1	0	1	1
1	1	0	1	1	0
1	1	1	0	1	0

2.[29 points] Programmable Machine

(a)[9 points] Below is the picture of a small processor and a FSM which computes

$$F = N + (N-1) + (N-2) + \dots + 2 + 1$$



Complete following tables

s	z	a_{SEL}	b_{SEL}	s'
00	0	11	00	01
00	1	11	00	01

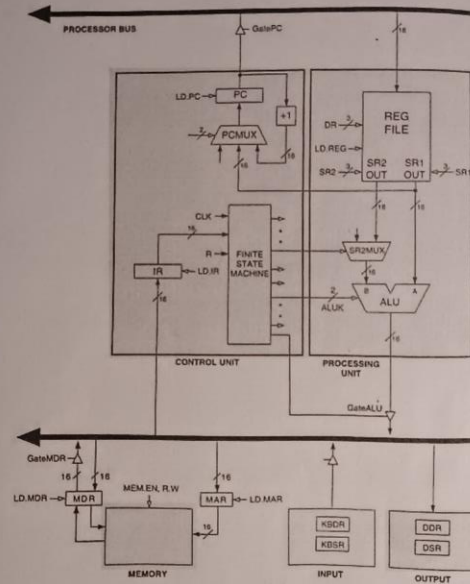
s	z	a_{SEL}	b_{SEL}	s'
01	0	01	01	01
01	1	01	01	10

s	z	a_{SEL}	b_{SEL}	s'
10	0	00	10	10
10	1	00	10	10

STUDENT NAME: Javohirbek Xatamov
 STUDENT ID NUMBER: U2210251 SECTION NUMBER IN THIS COURSE: 003

3.[9 points] LC-3 Architecture

Below is the picture of LC-3 processor



(a)[3 points] Fill in the values of control signals in the first instruction cycle? In that cycle, the following operations are processed at the same time.

MAR ← PC	GatePC	GateMDR	LD.REG	LD.IR	LD.PC	LD.MAR	LD.MDR
PC ← PC + 1	1	0	0	0	1	1	0

(b)[3 points] Fill in the values of control signals in the STORE RESULT cycle? The instruction is R3 ← R2 + R1.

GateALU	GateMDR	LD.REG	LD.IR	LD.PC	LD.MAR	DR
1	0	1	0	0	0	011

(c)[3 points] Fill in the values of control signals in the FETCH OPERAND cycle? The instruction is LDR R1 R2 #0

GateALU	GateMDR	LD.REG	LD.IR	LD.PC	LD.MAR	LD.MDR
0	0	0	0	0	0	1

STUDENT NAME: Tavohribe Xatamaw
 STUDENT ID NUMBER: U2210251 SECTION NUMBER IN THIS COURSE: 003

4. [24 points] LC-3 Machine Language

LD (Load) $0 \ 0 \ 1 \ 0 \ DR \ PCoffset9$ $DR = M(PC + SEXT(PCoffset9))$	ST (Store) $0 \ 0 \ 1 \ 1 \ SR \ PCoffset9$ $M(PC + SEXT(PCoffset9)) = SR$	LEA (Load Effective Address) $1 \ 1 \ 1 \ 0 \ DR \ PCoffset9$ $DR = PC + SEXT(PCoffset9)$	BR (Conditional Branch) $0 \ 0 \ 0 \ 0 \ n \ p \ PCoffset9$ $if ((N \ AND \ n) \ OR \ (Z \ AND \ 1) \ OR \ (P \ AND \ p))$ $PC = PC + SEXT(PCoffset9)$
LI (Load Indirect) $1 \ 0 \ 1 \ 0 \ DR \ PCoffset9$ $DR = M(M(PC + SEXT(PCoffset9)))$	STI (Store Indirect) $1 \ 0 \ 1 \ 1 \ SR \ PCoffset9$ $M(M(PC + SEXT(PCoffset9))) = SR$	ADD (Addition) $0 \ 0 \ 0 \ 1 \ DR \ SR1 \ SR2$ $DR = SR1 + SR2$	AND (Bit-wise Logical AND) $0 \ 1 \ 0 \ 1 \ DR \ SR1 \ SR2$ $DR = SR1 \ AND \ SR2$
LDR (Load Base + offset) $0 \ 1 \ 1 \ 0 \ DR \ BaseR \ offset6$ $DR = M(BaseR + SEXT(offset6))$	STR (Store Base + offset) $0 \ 1 \ 1 \ 1 \ SR \ BaseR \ offset6$ $M(BaseR + SEXT(offset6)) = SR$	NOT (Load Base + offset) $1 \ 1 \ 0 \ 1 \ DR \ SR1$ $DR = NOT(SR1)$	

(a) [8 points] Finish LC-3 machine code to load R1 with integer 224 (R1 <- 224). Assume that your memory and registers are empty, and try to make it run as fast as possible. Write semantics of your instructions to the right of your instructions.

0 1 0 1 0 0 1 0 0 1 0 0 0 0 0 0	R1 <- 0
0 0 0 1 0 0 1 0 0 1 1 0 1 1 1 1	R1 <- R1 + 15 R1 = 15
0 0 0 1 0 0 1 0 0 1 0 0 0 0 0 1	R1 <- R1 + R1 R1 = 30
0 0 0 1 0 0 1 0 0 1 0 0 0 0 0 1	R1 <- R1 + R1 R1 = 60
0 0 0 1 0 0 1 0 0 1 0 0 0 0 0 1	R1 <- R1 + R1 R1 = 120
0 0 0 1 0 0 1 0 0 1 0 0 0 0 0 1	R1 <- R1 + R1 R1 = 240
0 0 0 1 0 0 1 0 0 1 0 0 0 0 0 0	R1 <- R1 - 16 R1 = 224

(b) [8 points] Now, R1 has integer 224 and integer k is stored at location x30FF in memory. Finish LC-3 machine code to calculate 224*k and save the result to R0. Write semantics of your instructions to the right of your instructions. Your instructions start at location x3000.

x3000	0 1 0 1 0 1 1 0 1 1 0 0 0 0 0 0	R3 <- 0
x3001	0 0 0 0 0 1 0 0 1 1 1 1 1 1 0 1	R2 <- M[x30FF]
x3002	0 0 0 0 0 1 0 0 0 0 0 0 0 1 1 1	if (R2 == 0) Branch out of loop
x3003	0 0 0 1 0 0 1 1 0 1 1 0 0 0 0 1	R5 <- R3 + R1
x3004	0 0 0 1 0 0 1 1 0 1 1 1 1 1 1 1	R7 <- R2 - 1
x3005	0 0 0 0 0 1 1 1 1 1 1 1 1 1 0 0	Branch to start of loop
x3006	0 0 0 1 0 0 0 0 0 1 1 1 0 0 0 0	R0 <- R3 + 0

(c) [8 points] Above code may be inefficient when integer k is bigger than 224. Write machine code to make it more efficient by comparing 224 with k. Later you're going to insert this machine code between instructions at x3001 and x3002 without updating your code in problem (b)

x3002	1 0 0 1 1 0 1 1 1 1 1 1 1 1 1 1	R4 <- R1 - R2
x3003	0 0 0 1 1 0 1 1 1 1 1 1 1 1 1 1	(Use R5 to store the 2's complement of R2)
x3004	0 0 0 1 1 0 0 0 1 1 1 1 1 1 1 1	
x3005	0 0 0 0 0 1 1 1 0 0 0 0 0 0 1 1	if (R4 >= 0) Branch to x3009
x3006	0 0 0 1 1 0 1 1 0 0 1 1 0 0 0 0	Swap R1 and R2
x3007	0 0 0 1 1 0 1 1 0 0 1 1 0 0 0 0	(Use R5 for storing the temporary value)
x3008	0 0 0 1 1 0 1 1 0 0 1 1 0 0 0 0	
x3009	0 0 0 1 1 0 1 1 0 0 1 1 0 0 0 0	R2 <- R2 + 0

STUDENT NAME: Jawahirbek Xatamov
 STUDENT ID NUMBER: U2210251 SECTION NUMBER IN THIS COURSE: 003

5.[16 points] LC-3 Machine Language

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
x30F6	1	1	1	0	0	0	1	1	1	1	1	1	0	1		
x30F7	0	0	0	1	0	1	0	0	0	1	1	0	1	1	0	
x30F8	0	0	1	1	0	1	0	1	1	1	1	1	0	1	1	
x30F9	0	1	0	1	0	1	0	0	1	0	1	0	0	0	0	
x30FA	0	0	0	1	0	1	0	0	1	0	1	0	0	1	0	
x30FB	0	1	1	1	0	1	0	0	0	1	0	0	1	1	0	
x30FC	1	0	1	0	0	1	1	1	1	1	1	0	1	1	1	

(a)[4 points] The PC has just become x30F8 at the Fetch phase. What is the value of R1?

R1 = x30F4

(b)[4 points] The PC has just become x30F9 at the Fetch phase. What is the value of R2?

R2 = x1102

(c)[4 points] The PC has just become x30FB at the Fetch phase. What is the value of R2?

R2 = 0

(d)[4 points] The PC has just become x30FC at the Fetch phase. What is the value of R2?

R2 = 5

6.[10 points] ISA and ISA Trade-offs

(a)[10 points] Circle Uarch or ISA corresponding to whether each of the following characteristics is better classified as "microarchitecture" or "ISA":

Size of the general purpose registers: Uarch / ISA

Load/store vs. memory/memory: Uarch / ISA

Support for virtual memory: Uarch / ISA

The number of ALU's in the CPU: Uarch / ISA

The number of general purpose registers in the CPU: Uarch / ISA

Variable length instructions: Uarch / ISA

Exception and interrupt handling: Uarch / ISA

Access protection: Uarch / ISA

Privilege modes: Uarch / ISA

Special I/O instructions: Uarch / ISA